

# 5COSC019W Object Oriented Programming

## Tutorial 1 Lab Guide — University Lab PC

|                          |                                                                   |
|--------------------------|-------------------------------------------------------------------|
| <b>Module Code</b>       | 5COSC019W                                                         |
| <b>Module Title</b>      | Object Oriented Programming                                       |
| <b>Level</b>             | Level 5 — 20 Credits                                              |
| <b>Session</b>           | Tutorial 1 Lab — 2 hours                                          |
| <b>Location</b>          | University computer lab — IntelliJ IDEA launched via AppsAnywhere |
| <b>Learning Outcomes</b> | LO1, LO2, LO5                                                     |

### PURPOSE

By the end of this lab you will have a Spring Boot application running, a documented API you can test in the browser, and a GitHub repository with a meaningful commit history.

### IMPORTANT — YOU ARE ON A SHARED, NON-PERSISTENT MACHINE

This lab runs on a university PC, not your own laptop, and IntelliJ IDEA is streamed to you through AppsAnywhere. Two things follow from that, and they change how you should work today:

1. Local files are not guaranteed to survive after you log out. Treat GitHub as your real save button. Commit and push after every activity, not just at the end.
2. This machine will be used by someone else after you. Sign out of every account (GitHub, IntelliJ) before you leave — see the Wrap-up section.

## Before You Arrive

Make sure you have the following ready before the session starts — there is no time built into this 2-hour lab to set these up from scratch.

- A personal GitHub account. Create one free at [github.com](https://github.com) if you don't already have one, and know your username.
- Your GitHub password remembered — you will log in through a browser window, not type it into IntelliJ directly.
- Your university username and password, for logging into the lab PC and AppsAnywhere.
- Recommended: know how to reach your university OneDrive or network drive (often an H: drive) as a second safety net alongside GitHub.

## Session Timing at a Glance

| Time        | Activity                                                                          |
|-------------|-----------------------------------------------------------------------------------|
| 0:00 – 0:15 | Activity 1: Launch AppsAnywhere, sign in to GitHub, clone the starter project     |
| 0:15 – 0:25 | Activity 1 (cont.): Run the application; verify /hello and /status in the browser |
| 0:25 – 0:45 | Activity 2: Add Swagger UI and verify it in the browser                           |

| Time        | Activity                                                           |
|-------------|--------------------------------------------------------------------|
| 0:45 – 1:05 | Activity 3: Extend the application — new field, new endpoint       |
| 1:05 – 1:35 | Activity 4: Git commit & push to GitHub                            |
| 1:35 – 1:55 | Activity 5: Stretch task (optional, for students who finish early) |
| 1:55 – 2:00 | Wrap-up: submit your link, sign out of everything, log off         |

## Learning Objectives for This Lab

- Launch and run a Spring Boot application on a managed university PC via AppsAnywhere.
- Navigate Swagger UI and test endpoints without Postman.
- Modify an existing Java class to add a new field, following the JavaBean convention.
- Write a new `@GetMapping` endpoint and verify it appears in Swagger UI automatically.
- Make a meaningful Git commit and push it to a personal GitHub repository, using an authentication method appropriate for a shared machine.

## Activity 1 — Launch, Sign In & Clone (25 min)

Goal: get IntelliJ open, connected to your GitHub account, and running the starter project — without typing any long-term passwords into a shared machine.

| # | Student Task                                                                                                                                                                                                          | Success Indicator                                                                          |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1 | Log in to the university PC with your university credentials.                                                                                                                                                         | Desktop loads normally.                                                                    |
| 2 | Open AppsAnywhere (desktop icon or portal link).<br>Find Git and click Launch.<br>Find IntelliJ IDEA and click Launch. The first launch can take a few minutes while it streams to the PC — be patient!               | IntelliJ IDEA opens on screen.                                                             |
| 3 | Download from BB products-api.zip (Learning Resources → Week 02 → Tutorial 1) and unzip it.                                                                                                                           | IntelliJ shows the project.                                                                |
| 4 | Inside IntelliJ:<br>In the Welcome page → Open → Select the pom.xml file → Open as a Project (Trust Project)<br>Right click on pom.xml file<br>Wait for the bar on the bottom of the IDE to load all the dependencies | In the project folder under “External Libraries” you should see all the Maven dependencies |
| 5 | Open ProductsApiApplication.java (inside products-api → src → main → java) and click the green Run button (or press Shift+F10). Watch the console.                                                                    | Console shows: Started ProductsApiApplication                                              |
| 6 | Open a browser and go to <a href="http://localhost:8080/hello">http://localhost:8080/hello</a>                                                                                                                        | Browser shows: Hello from Spring Boot!                                                     |
| 7 | Also try <a href="http://localhost:8080/status">http://localhost:8080/status</a>                                                                                                                                      | Browser shows the status message.                                                          |

### TROUBLESHOOTING — ACTIVITY 1

AppsAnywhere is slow or seems stuck: give it a couple of minutes on first launch before asking for help — streaming a full IDE takes longer than opening a normal app.

If you do not see the Maven libraires under “External Libraries”: Right click on pom.xml → Maven → Sync Project

## Activity 2 — Add Swagger UI (20 min)

Goal: make your API visible and testable in the browser, without needing Postman.

| #  | Student Task                                                                                                                                                                                               | Success Indicator                                                                           |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 8  | Open pom.xml. Inside the <dependencies> section, add the springdoc-openapi dependency block shown below. Save the file.<br><br>If the section appear in red: Right click on pom.xml → Maven → Sync Project | Maven re-imports automatically (bottom bar).<br>Section present in the pom.xml file not RED |
| 9  | Stop the application (red square) and restart it (green play). Wait for Started in the console.                                                                                                            | Console shows the application started successfully.                                         |
| 10 | Open the browser and go to <a href="http://localhost:8080/swagger-ui.html">http://localhost:8080/swagger-ui.html</a>                                                                                       | Swagger UI loads, showing /hello and /status.                                               |
| 11 | Click GET /hello, then Try it out, then Execute. Look at the response body, status code (200), and headers.                                                                                                | Response body shows: Hello from Spring Boot!                                                |

Dependency block to add inside <dependencies> in pom.xml:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.1.1</version>
</dependency>
```

### TROUBLESHOOTING — ACTIVITY 2

If pom.xml shows a red underline on the new dependency, Maven could not resolve it — usually a network issue on the university connection. Try the Maven "reload" icon (circular arrows) in the Maven tool window on the right-hand sidebar first or Right click on pom.xml → Maven → Sync Project

## Activity 3 — Extend the Application (20 min)

Goal: practise modifying an existing class and adding a new endpoint — exactly the skill the lab-based practical in Lecture 8 will assess.

| #  | Student Task                                                                                                                                               | Success Indicator                                                 |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| 12 | Open Person.java. It currently has a name field. Add a new private String field called email, plus a getter following the JavaBean convention: getEmail(). | Person.java compiles with no errors.                              |
| 13 | Open HelloController.java. Add a new endpoint: @GetMapping("/goodbye") returning the String "Goodbye from Spring Boot!"                                    | Method added; application restarts without errors.                |
| 14 | Restart the application and refresh Swagger UI.                                                                                                            | /goodbye now appears alongside /hello and /status, automatically. |
| 15 | Use Try it out on /goodbye in Swagger UI to confirm the response.                                                                                          | Response body matches exactly what you returned.                  |

**COMMON MISTAKE**

If you add an endpoint or field and Swagger UI doesn't show the change, you almost certainly forgot to restart the application. Spring Boot does not pick up code changes automatically unless Spring DevTools is configured — it isn't this week.

## Activity 4 — Git Commit & Push (30 min)

Goal: save your work permanently to GitHub. This is longer than a typical Git activity because on a shared machine, authentication is the step most likely to need a second attempt — build in the time now rather than at 1:55.

| #  | Student Task                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Success Indicator                                                                                         |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 16 | In task 2, you launched Git from AppsAnywhere. Check that is running:<br>Open the terminal in IntelliJ (from the bottom left bar tools) and run <code>git --version</code>                                                                                                                                                                                                                                                                                                        | Terminal prompt the git version                                                                           |
| 17 | Login in your GitHub from IntelliJ:<br>In IntelliJ: Settings → Version Control → GitHub → Add account... → Log In with GitHub<br>Click on Apply and Ok                                                                                                                                                                                                                                                                                                                            | You should see your GitHub account in the IntelliJ window under Version Control                           |
| 18 | Set your identity:<br>From the terminal in IntelliJ run:<br><code>git config --global user.name "GitHub username"</code><br><code>git config --global user.email "GitHub email"</code><br><br>If your email in GitHub is private:<br>Settings → Emails. If 'Keep my email address private' is ticked — it is by default — you must use the noreply address shown there as their git email. In this case re-run the <code>git config --global user</code> with this noreply-email. | In the terminal, if you run:<br><code>git config -global -list</code><br>you will see the identity is set |
| 19 | Share the project:<br>Select “VCS” from the top bar menu → Enable VersionControl Integration → Select Git and Enable.<br>Then Git should appear on the top bar menu.<br>From the top bar menu: Git → GitHub → Share Project on GitHub (untick Private)<br><br>Write an initial commit, check the list and press “Add”                                                                                                                                                             | You receive a message on the bottom right corner: Successfully shared project on Github                   |
| 20 | Verify on GitHub: open the repo in a browser: <code>src/</code> and <code>pom.xml</code> should be there                                                                                                                                                                                                                                                                                                                                                                          | Your latest changes are visible on GitHub.                                                                |

## Activity 5 — Stretch Task (20 min, Optional)

**OPTIONAL**

This task is for students who finish early. It is not collected or marked — it's practice, and a good chance to try something without a numbered step-by-step.

- Add a third endpoint, GET `/status`, that returns a message including today's date — use Java's `LocalDate.now().toString()`.

- Create a second controller class called InfoController with an endpoint GET /info that returns a String describing the application.
- Refresh Swagger UI and confirm both new endpoints appear, grouped under their respective controllers.
- Commit your changes: "Tutorial 1: stretch task — status endpoint with date, InfoController added", and push.

## Wrap-Up (5 min) — Read This Before You Leave

---

21. Paste your GitHub repository URL into the Blackboard submission link. This is today's deliverable.
22. Sign out of GitHub in the browser: click your avatar (top right) → Sign out.
23. Close IntelliJ and log off the university PC completely. The next student should see a clean login screen, not your session.

### KEY POINT

This GitHub repository is part of the 60% coursework. A repo with one commit on submission day looks very different from one with 11 weeks of real history — keep this in mind!

## Appendix — Reference Material

---

### What Your Finished Files Should Look Like

Use this to check your own work after Activity 3 — try to write it yourself first.

#### Person.java

```
package uk.ac.westminster.productsapi;

public class Person {

    private String name;
    private String email;

    public Person() {}

    public Person(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

#### HelloController.java

```
package uk.ac.westminster.productsapi;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot!";
    }

    @GetMapping("/goodbye")
    public String goodbye() {
        return "Goodbye from Spring Boot!";
    }

    @GetMapping("/status")
    public String status() {
        return "API is running – Tutorial 1";
    }
}
```

### Recommended Resources

- Spring Initializr: [start.spring.io](https://start.spring.io) — the official project generator
- springdoc-openapi: [springdoc.org](https://springdoc.org) — documentation and configuration reference
- Baeldung: [baeldung.com](https://baeldung.com) — search "Spring Boot Hello World" for a detailed tutorial

- Git documentation: [git-scm.com/doc](https://git-scm.com/doc) — official reference
- GitHub Student Pack: [education.github.com/pack](https://education.github.com/pack) — free tools including GitHub Pro

## Assessment Alignment

| Outcome | Tutorial 1 Evidence                                                                           | Contributes To                         |
|---------|-----------------------------------------------------------------------------------------------|----------------------------------------|
| LO1     | First use of classes and objects in Java; Person class with a properly encapsulated field     | Enterprise Application                 |
| LO2     | Spring Boot application running; REST endpoints via @GetMapping; springdoc library integrated | Enterprise Application                 |
| LO5     | GitHub repository created; first meaningful commit pushed — this is VCS evidence              | Enterprise Application + Lab Practical |